
Distributed Systems

Monsoon 2025

Lecture 14b

International Institute of Information Technology
Hyderabad, India

Resources:

1. The book Distributed Systems: Concepts and Design, Kindberg et al.
2. The paper Efficient Commit Protocols for the Tree of Processes Model of Distributed Transactions, C. Mohan, B. Lindsay, 1983, ACM
3. Lecture slides by Kathleen Durant, NEU.

Transactions

- Some situations, clients require a sequence of requests to a server to be **atomic**.
- **Atomic** understood as free from interruption by operations being performed on behalf of other concurrent clients.
- Either all of the operations must be completed successfully or they must have no effect at all in the presence of server crashes.

Transactions

- The goal of transactions is to ensure that all of the objects managed by a server remain in a **consistent** state when they are accessed by multiple transactions even in the presence of server crashes.
- Transactions deal with crash failures of processes and omission failures in communication, but **not** any type of **arbitrary** (or Byzantine) behaviour.

Fault Models

- Lampson proposed a fault model for distributed transactions that accounts for failures of disks, servers and communication.

ACID

- The properties of transactions are:
- **Atomicity**

ACID

- The desirable properties of transactions are:
- **Atomicity**: a transaction must be all or nothing;

ACID

- The desirable properties of transactions are:
- **Atomicity**: a transaction must be all or nothing;
- **Consistency**:

ACID

- The desirable properties of transactions are:
- **Atomicity**: a transaction must be all or nothing;
- **Consistency**: a transaction takes the system from one consistent state to another consistent state;

ACID

- The desirable properties of transactions are:
- **Atomicity**: a transaction must be all or nothing;
- **Consistency**: a transaction takes the system from one consistent state to another consistent state;
- **Isolation**:

ACID

- The desirable properties of transactions are:
- **Atomicity**: a transaction must be all or nothing;
- **Consistency**: a transaction takes the system from one consistent state to another consistent state;
- **Isolation**: Each transaction must be performed without interference from other transactions;

ACID

- The desirable properties of transactions are:
- **Atomicity**: a transaction must be all or nothing;
- **Consistency**: a transaction takes the system from one consistent state to another consistent state;
- **Isolation**: Each transaction must be performed without interference from other transactions;
- **Durability**:

ACID

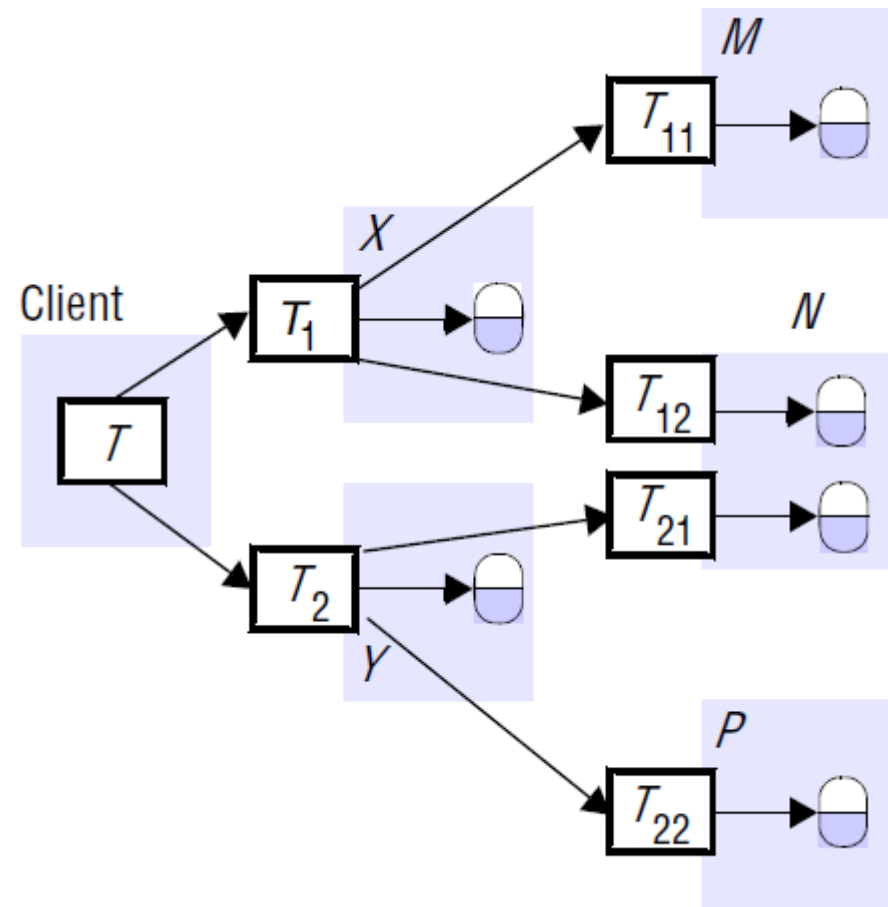
- The desirable properties of transactions are:
- **Atomicity**: a transaction must be all or nothing;
- **Consistency**: a transaction takes the system from one consistent state to another consistent state;
- **Isolation**: Each transaction must be performed without interference from other transactions;
- **Durability**: After a transaction has completed successfully, all its effects are saved in permanent storage
- Härder and Reuter suggested the mnemonic '**ACID**' to remember these properties.

ACID

- A transaction is either **successful** or is **aborted** in one of two ways – the client aborts it or the server aborts it.

Nested Transactions

- Transactions can be **nested** too.
- Figure shows an example.
- Semantics can be extended to nested transactions also.



Nested Transactions

- An example setting for nested transactions is as follows.
- Consider booking a holiday on a website such as makemytrip.com.
- The holiday may involve multiple entities: flights, ground transport, meals, adventure activities, hotel, and so on.
- Each is typically handled by different agencies and may involve further hierarchical steps.

Nested Transactions

- The **rules** for committing of nested transactions are:
- A transaction may commit or abort only after its child transactions have completed.
- When a subtransaction completes, it makes an independent decision either to commit provisionally or to abort. Its decision to abort is final.
- When a parent aborts, all of its subtransactions are aborted.
- When a subtransaction aborts, the parent can decide whether to abort or not.
- If the top-level transaction commits, then all of the subtransactions that have provisionally committed can commit too, provided that none of their ancestors has aborted.

Means of Handling Concurrency

- Locks
- Time stamp ordering
- Optimistic Concurrency control
- Read more offline.
 - Distributed Systems: Concepts and Design, Kindberg et al. has a chapter on database systems and transactions

On to Distributed Transactions

- A *distributed transaction* refers to a flat or nested transaction that accesses objects managed by **multiple servers**.
- When a distributed transaction comes to an end, the atomicity property of transactions requires that either **all of the servers** involved in the transaction **commit** the transaction or **all of them abort** the transaction.
- To achieve this, one of the servers acts as a **coordinator** to ensure the same outcome at all of the servers.

On to Distributed Transactions

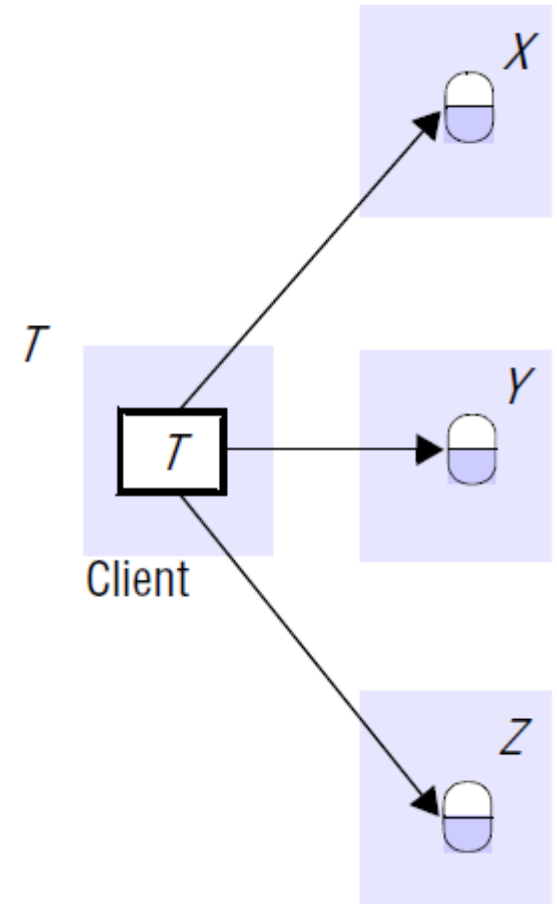
- A quick example of a distributed transaction is as follows.
- Imagine that you want to pay for your cup of tea at the canteen using an UPI application.
- The canteen vendor may have his phone number linked with his bank with Union Bank.
- Your phone number is linked with your account held at, say SBI.
- The transaction has to debit your account and credit to the canteen vendor account.

Distributed Transactions

- The actions of the coordinator are guided by the protocol chosen.
- A protocol known as the ‘two-phase commit protocol’ is the most commonly used.
- This protocol allows the servers to communicate with one another to reach a joint decision as to whether to commit or abort.

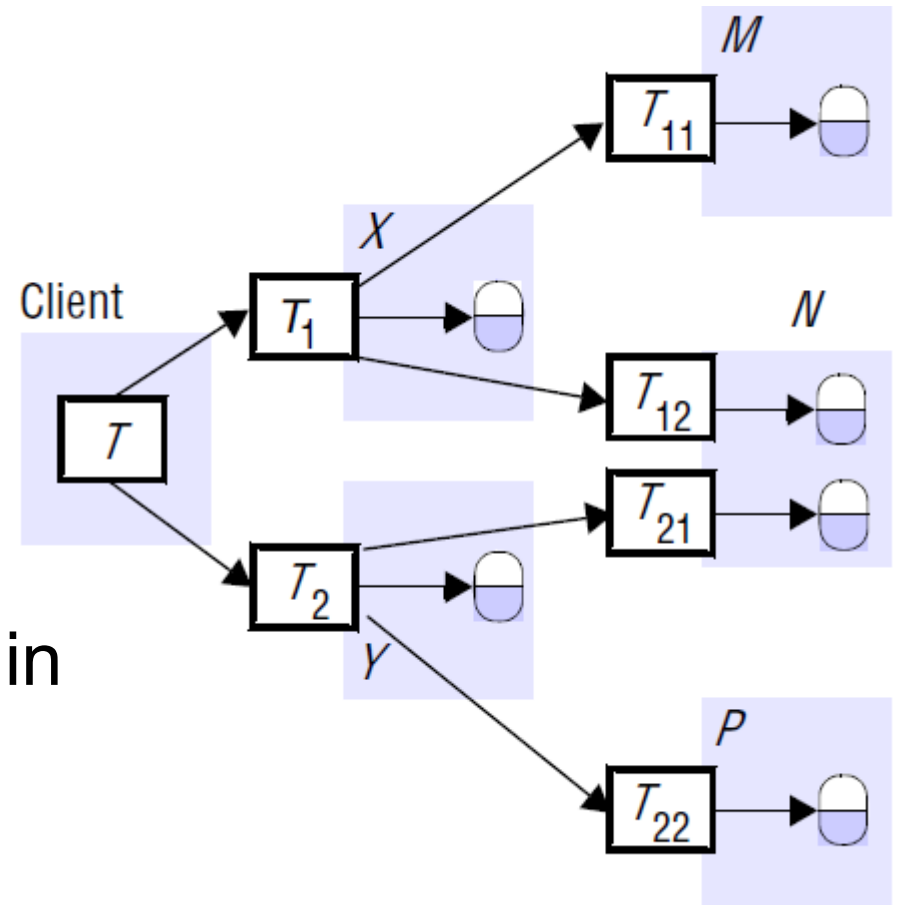
Flat vs Nested Transactions

- In a flat transaction, a client makes requests to more than one server.
- A flat client transaction **completes** each of its requests **before** going on to the next one.
- Therefore, each transaction accesses servers' objects sequentially.
- When servers use locking, a transaction can only be waiting for one object at a time.



Flat vs Nested Transactions

- In a nested transaction, the top-level transaction can initiate **subtransactions**, and each subtransaction can initiate further subtransactions.
- **Subtransactions** at the same level can run concurrently.
- So T_1 and T_2 are concurrent, and as they invoke objects in different servers, they can run in parallel.
- The four subtransactions T_{11} , T_{12} , T_{21} and T_{22} also run concurrently.



Distributed Systems

Monsoon 2025

Lecture 15

International Institute of Information Technology
Hyderabad, India

Resources:

1. The book Distributed Systems: Concepts and Design, Kindberg et al.
2. The paper Efficient Commit Protocols for the Tree of Processes Model of Distributed Transactions, C. Mohan, B. Lindsay, 1983, ACM
3. Lecture slides by Kathleen Durant, NEU.

The Coordinator and the Participant

- Servers that execute requests as part of a distributed transaction need to be able to communicate with one another to coordinate their actions when the transaction commits.
- A client starts a transaction by sending an *openTransaction* request to a coordinator.
- The coordinator carries out the *openTransaction* and returns the resulting transaction identifier (TID) to the client.
- Transaction identifiers for distributed transactions must be unique within a distributed system.

The Coordinator and the Participant

- The **coordinator that opened** the transaction becomes the coordinator for the distributed transaction and at the end **is responsible for committing or aborting it.**
- Each of the servers that manages an object accessed by a transaction is a participant in the transaction and provides an object to the *participant*.
- The participants are responsible for **cooperating** with the coordinator in carrying out the commit protocol.

Atomic commit protocols

- Transaction commit protocols are from the early 1970s
- The atomicity property of transactions requires that when a **distributed** transaction comes to an end, either all of its operations are carried out or none of them.
- In the case of a distributed transaction, the client has requested operations at more than one server.

Atomic commit protocols

- A transaction comes to an end when the client requests that it be committed or aborted.
- A simple way to complete the transaction in an atomic manner is for the coordinator to communicate the commit or abort request to all of the participants in the transaction and to keep on repeating the request until all of them have acknowledged that they have carried it out.
- This is called as *one phase atomic commit protocol*.

Atomic Commit Protocols

- This simple one-phase atomic commit protocol is **inadequate**.
- It does not allow a server to make a unilateral decision to abort a transaction when the client requests a commit.
- Reasons that prevent a server from being able to commit its part of a transaction generally relate to issues of concurrency control.
 - For example, the resolution of a deadlock can lead to the aborting of a transaction without the client being aware
 - optimistic concurrency control is in use, the failure of validation at a server would cause it to decide to abort the transaction.
 - Finally, the coordinator may not know if a server has crashed and been replaced during the progress of a distributed transaction

Atomic Commit Protocols

- The *two-phase commit protocol* is designed to allow any participant to abort its part of a transaction.

Atomic Commit Protocols

- In the first phase of the protocol, each participant votes for the transaction to be committed or aborted.
- Once a participant has voted to commit a transaction, it is not allowed to abort it.
- Therefore, before a participant votes to commit a transaction, it must ensure that it will eventually be able to carry out its part of the commit protocol, even if it fails and is replaced in the interim.
- A participant in a transaction is said to be in a *prepared* state for a transaction if it will eventually be able to commit it.
- To make sure of this, each participant saves in permanent storage all of the objects that it has altered in the transaction, together with its status – prepared.

Atomic Commit Protocols

- In the **second phase** of the protocol, every participant in the transaction carries out the joint decision.
- If **any one participant votes to abort**, then the decision can be to **abort** the transaction.
- If **all the participants vote to commit**, then the decision can be to **commit** the transaction.

Atomic Commit Protocols

- The problem is to ensure that all of the participants vote and that they all reach the same decision.
- This is fairly simple if no errors occur, but the protocol must work correctly even when some of the servers fail, messages are lost or servers are temporarily unable to communicate with one another.

Atomic Commit Protocols

- Commit protocols are designed to work in an **asynchronous** system in which
 - servers may crash and
 - messages may be lost.
 - It is assumed that an underlying request-reply protocol removes corrupt and duplicated messages.
- There are **no Byzantine faults**: servers either crash or obey the messages they are sent.

Atomic Commit Protocols

- The two-phase commit protocol is an example of a protocol for reaching a consensus.
- Recall that consensus cannot be reached in an asynchronous system if processes sometimes fail.
- However, the two-phase commit protocol does reach consensus under those conditions.
- This is because processes that crash are assumed to be replaced with a new process.
- The *new* process has its state information set from the state saved in permanent storage and information held by other processes.

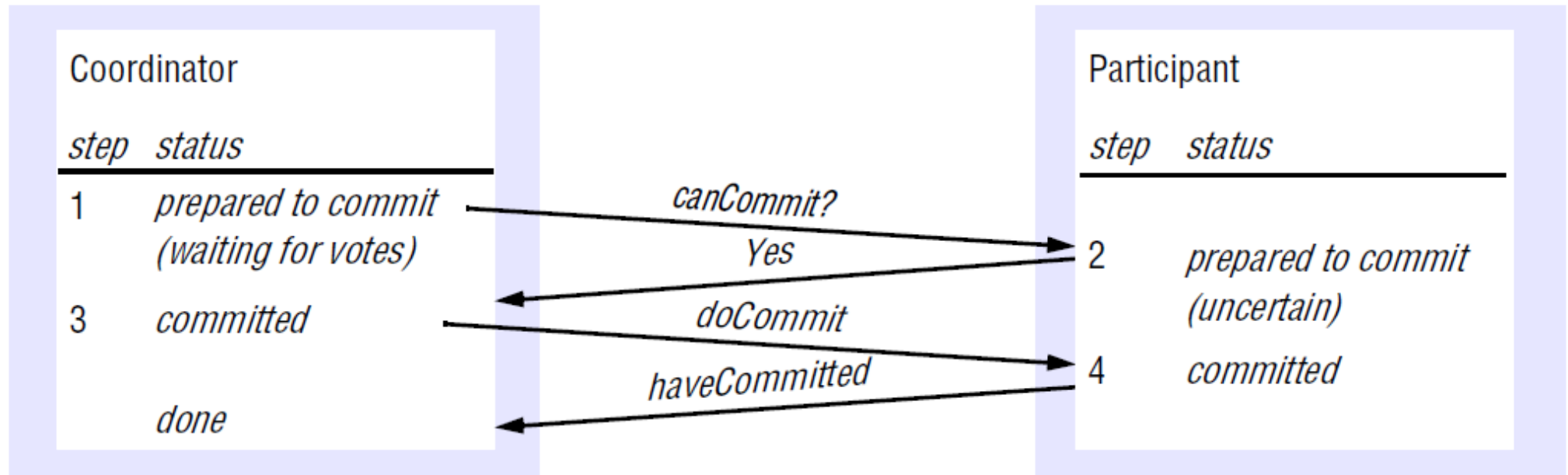
Two-Phase Commit Protocol

- *Phase 1 (voting phase):*
 1. The coordinator *sends a `canCommit?`* request to each of the participants in the transaction.
 2. When a participant receives a *`canCommit?`* request it *replies with its vote* (Yes or No) to the coordinator.
 3. Before voting *Yes*, it prepares to commit by saving objects in permanent storage.
 4. If the vote is *No*, the participant aborts immediately.

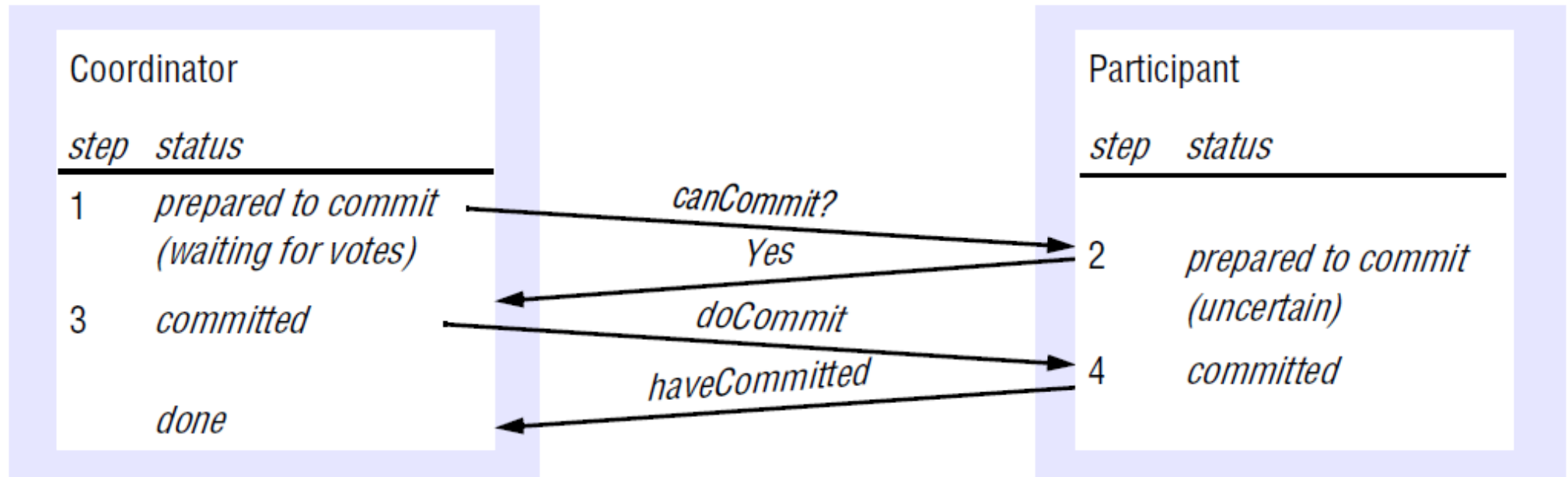
Two-Phase Commit Protocol

- *Phase 2* (completion according to outcome of vote):
- The coordinator **collects** the votes (including its own).
 - If there are no failures and **all the votes are Yes**, the coordinator decides to **commit** the transaction and sends a **doCommit** request to each of the participants.
 - **Otherwise**, the coordinator decides to abort the transaction and sends **doAbort** requests to all participants that voted Yes.
- Participants that voted Yes are **waiting** for a *doCommit* or *doAbort* request from the coordinator.
- When a participant receives one of these messages it acts accordingly and, in the case of commit, makes a **haveCommitted** call as confirmation to the coordinator.

Two-Phase Commit Protocol

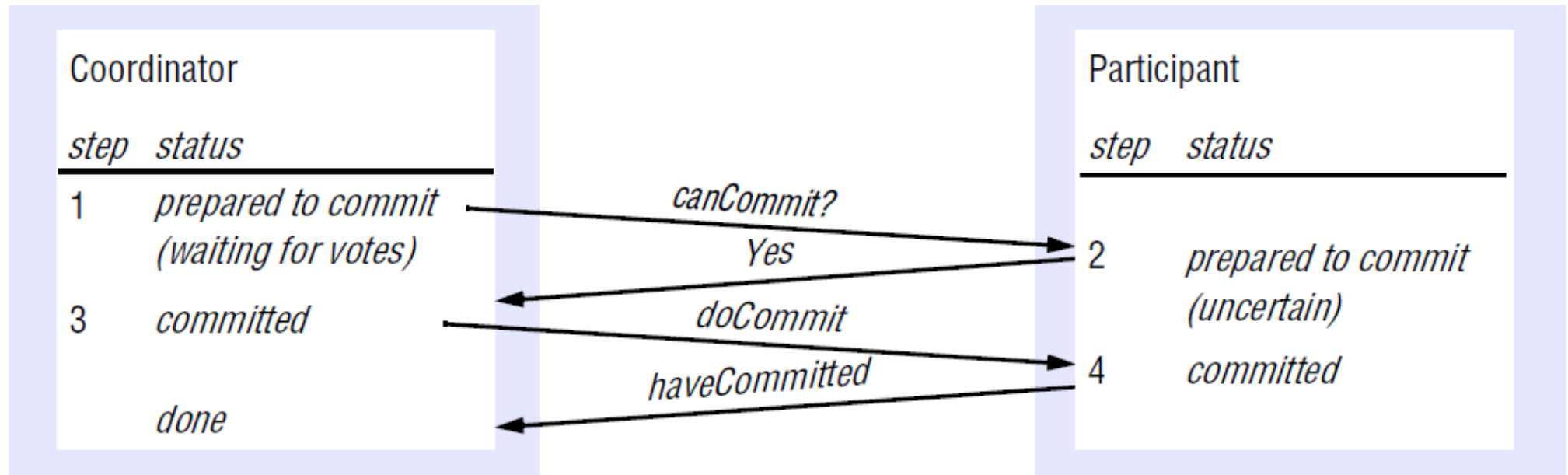


Two-Phase Commit Protocol



- Timeout situations
- Consider first the situation where a participant has **voted Yes** and is **waiting for the coordinator** to report on the outcome of the vote by telling it to commit or abort the transaction
- Such a participant is **uncertain** of the outcome and cannot proceed any further until it gets the outcome of the vote from the coordinator.

Two-Phase Commit Protocol



- The participant can make a *getDecision* request to the coordinator to determine the outcome of the transaction.
- When it gets the reply, it continues the protocol at step 4. If the coordinator has failed, the participant will not be able to get the decision until the coordinator is replaced, which can result in *extensive delays* for participants in the *uncertain* state.

Timeout Scenarios

- Another point at which a participant may be delayed is when it has carried out all its client requests in the transaction but has **not yet received a *canCommit?*** call from the coordinator.
- As the client sends the *closeTransaction* to the coordinator, a participant can only detect such a situation if it notices that it has not had a request in a particular transaction for a long time
- As no decision has been made at this stage, the participant can decide to ***abort unilaterally***

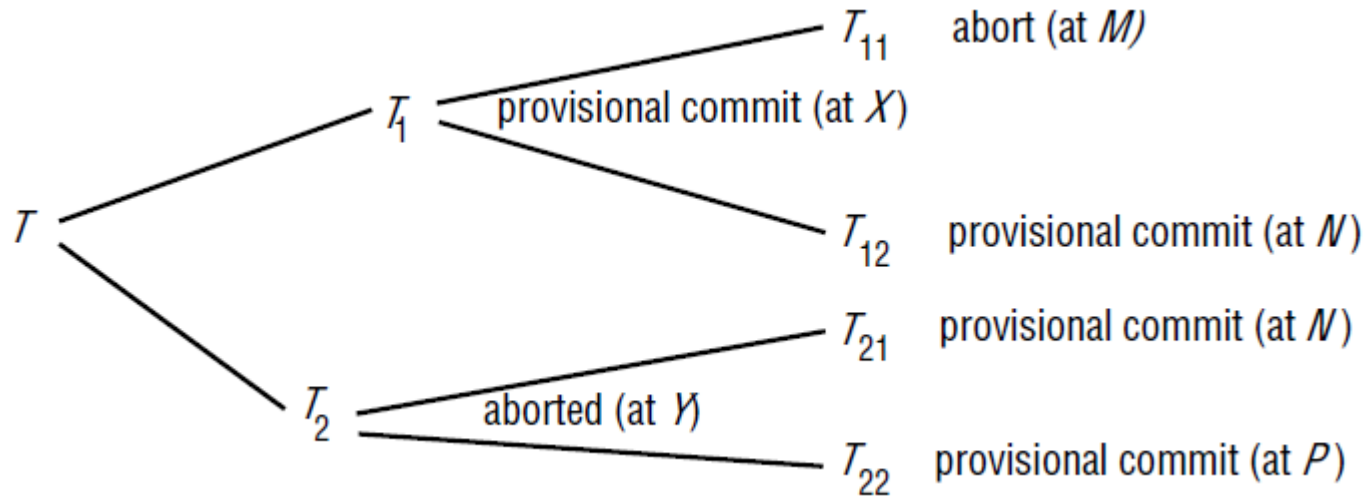
Timeout Scenarios

- The **coordinator may be delayed** when it is waiting for votes from the participants.
- As it has not yet decided the fate of the transaction it may **decide to abort** the transaction after some period of time.
- It must then **announce *doAbort*** to the participants who have already sent their votes.
- Some tardy participants may try to vote *Yes* after this, but their votes will be ignored and they will enter the *uncertain* state as described above.

Performance

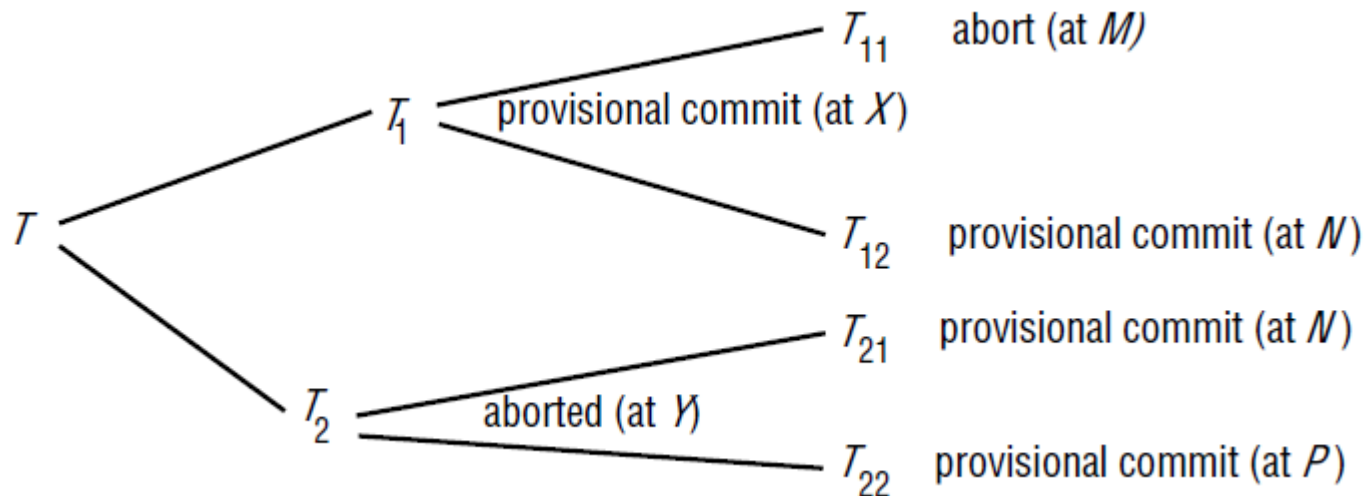
- The two-phase commit protocol involving N participants can be completed with N *canCommit?* messages and replies, followed by N *doCommit* messages.
- The cost in messages is proportional to $4N$, and the cost in time is four rounds of messages.
- In the worst case, there may be arbitrarily many server and communication failures during the two-phase commit protocol.
- However, the protocol is designed to **tolerate a succession of failures**.
- The protocol is guaranteed to **complete eventually**, although it is not possible to specify a time limit within which it will be completed.

2PC for Nested Transactions



- The outermost transaction in a set of nested transactions is called *the top-level transaction*.
- Transactions other than the top-level transaction are called *subtransactions*.
- For example, T_{11} and T_{12} start after T_1 and finish before it.

2PC for Nested Transactions



- When a subtransaction completes, it makes an independent decision either to commit provisionally or to abort.
- After all subtransactions have completed, the provisionally committed ones participate in a two-phase commit protocol, in which servers of provisionally committed subtransactions express their intention to commit and those with an aborted ancestor will abort.

Flat Vs. Hierarchical Models

- Hierarchical two-phase commit protocol
 - the two-phase commit protocol becomes a multi-level nested protocol.
 - The coordinator of the top-level transaction communicates with the coordinators of the subtransactions for which it is the immediate parent.
 - It sends *canCommit?* messages to each of the latter, which in turn pass them on to the coordinators of their child transactions (and so on down the tree).
- Each participant collects the replies from its descendants before replying to its parent.

Flat Vs. Hierarchical Models

- Flat 2-Phase Commit Protocol
 - the coordinator of the top-level transaction sends *canCommit?* messages to the coordinators of all of the subtransactions in the provisional commit list.

Other Aspects of Transactions

- Isolation
 - Use locks
 - Be careful about **distributed deadlocks**
- Recovery
 - Use logs to recover from various states in the 2Phase commit protocol.
 - Possible to extend the logging model to also nested transactions.

Few Optimizations

- Proposed by Mohan et al., there are two particular optimizations to the 2-Phase commit protocol.
- The **Presumed Abort** and the **Presumed Commit** are the two modifications.
- Consider completely or partially read-only transactions.
- A transaction is **partially read-only** if some processes of the transaction do not perform any updates to the data base, while the others do.
- A transaction is **(completely) read-only** if no process performs any updates.

Few Optimizations

- If a participant receives a PREPARE message and it finds that it has not done any updates, then it sends a READ VOTE, releases its locks, and "forgets" the transaction.
- The participant writes no log records.
- As far as it is concerned, it does not matter whether the transaction ultimately gets aborted or committed.
- So the participant, who is now known to the coordinator to be read-only, does not need to be sent COMMIT/ABORT message by the coordinator.
- There will not be a second phase of the protocol if the coordinator is read-only and gets only READ VOTES.

From Relational to Non-Relational Models

- Looser schema definition
- Applications written to deal with **specific type of data** such as documents
- Applications **aware of the schema definition** as opposed to the data
- Designed to handle **distributed, large databases**
- Trade offs:
 - No strong support for ad hoc queries but designed for **speed and growth of database**
 - Query language through the API
 - **Relaxation** of the ACID properties

RDBMS vs No-SQL

- **Elastic scaling**
 - RDBMS scale up – bigger load , bigger server
 - NoSQL scale out – distribute data across multiple hosts seamlessly
- **DBA Specialists**
 - RDBMS require highly trained expert to monitor DB
 - NoSQL require less management, automatic repair and simpler data models
- **Big Data**
 - Huge increase in data capacity and constraints of data volumes at its limits
 - NoSQL designed for big data

RDBMS vs No-SQL

- **Flexible data models**

- Change management to schema for RDBMS have to be carefully managed
- NoSQL databases more relaxed in structure of data
- Database schema changes do not have to be managed as one complicated change unit
- Application already written to address an amorphous schema

- **Economics**

- RDBMS rely on expensive proprietary servers to manage data
- No SQL: clusters of cheap commodity servers to manage the data and transaction volumes
- Cost per gigabyte or transaction/second for NoSQL can be lower than the cost for a RDBMS

RDBMS vs No-SQL Disadvantages

- **Support**

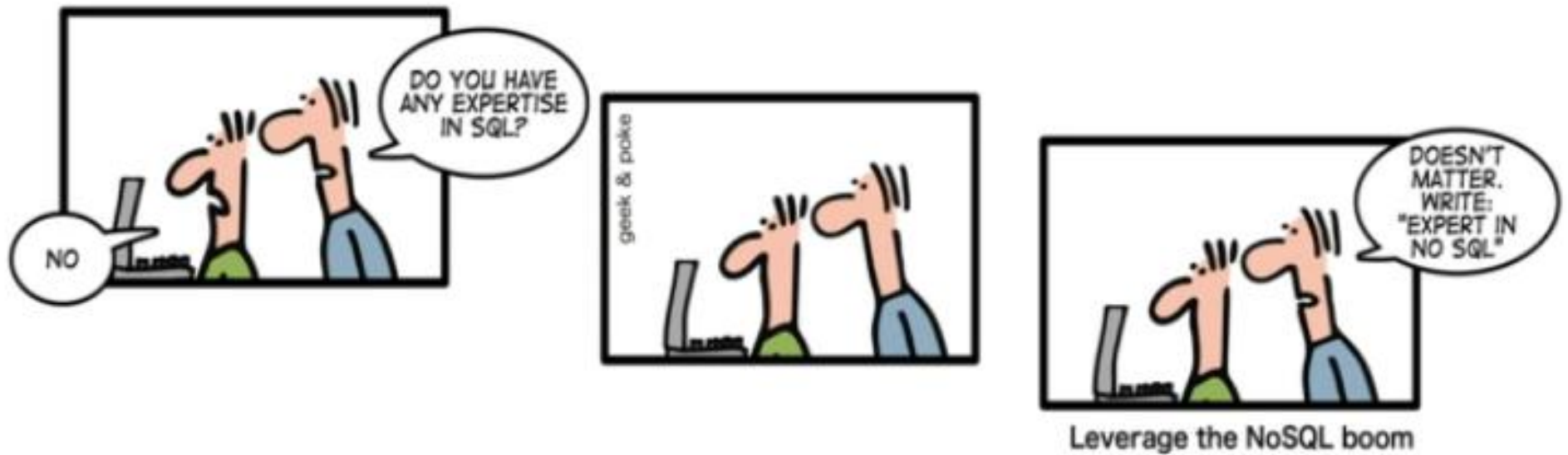
- RDBMS vendors provide a high level of support to clients
- Stellar reputation
- NoSQL –are open source projects with startups supporting them
- Reputation not yet established

- **Maturity**

- RDBMS a mature, stable and dependable product
- Also means no longer cutting edge
- NoSQL are still implementing their basic feature set

RDBMS vs No-SQL Disadvantages

HOW TO WRITE A CV



- **Lack of Expertise**
 - Whole workforce of trained and seasoned RDMS developers
 - Still recruiting developers to the NoSQL camp

RDBMS Vs NoSQL

- RDBMS follow the **ACID** model for transactions.
- A new name, called **BASE**, is what NoSQL databases follow.
- BASE stands for **Basically Available, Soft-state, Eventually consistent**.

Small Detour -- BASE

- Consider web applications that operate at large scale.
- One possibility to scale is the **vertical scaling** – operate out of the best computer available.
 - Not easy to maintain
- Another popular possibility is the **horizontal scaling** – operate across many computers.
- One question that then arises is to see how to **partition the data**. (database).
- **Two possibilities:**
 - Functional partitioning
 - Sharding

Small Detour

- In functional partitioning, data (tables) are grouped according to function.
- For instance, store user data in one place, e-transaction data in another place, inventory in another, and the like.
- In sharding, the entire database is divided into pieces and each piece is stored in one server.
- Functional partitioning can use sharding as a second level partitioning if the data for one function is too large, for instance.

Small Detour

- Consider a functional partition for the rest of this detour.
- We will see alternatives to ensuring consistency.
- In ACID style semantics, to maintain consistency in the functional partitioning model, one has to use the idea of **foreign keys**.
- We will see how we can move away from ACID semantics in the case of functional partitioned distributed database.
- Consider a simple schema of a bank tables for:
 - **UserProfile, eTransactions**
 - Users table stores data about each user including profile and rewards etc..
 - eTransactions has one record per transaction performed.

Small Detour

- We will see how we can move away from ACID semantics in the case of functional partitioned distributed database.
- Consider a simple schema of a bank tables for:
 - **UserProfile, eTransactions**
 - Users table stores data about each user including profile and rewards etc..
 - eTransactions has one record per transaction performed.

Detour

Begin Transaction

insert into eTransaction(tran_id, User, op, amount, description)

Update UserProfile set rewards += amount where user_id = user

Update

End Transaction

- We will show that it is easier to **relax consistency across functional groups** than within functional groups.
- Each time a transaction is done, a row is added to the transaction table and rewards for the user are updated in the Users table.
- Using an ACID-style transaction, the SQL would be as shown.

Detour

- Given this, the constraint on consistency could be relaxed.
- The user rewards can be set so that they **do not reflect the result of a transaction immediately**.
- This is not uncommon, and in fact people encounter this delay between a transaction and their rewards balance.

Detour

Begin transaction

insert into eTransaction(tran_id, User, op, amount, description)

End transaction

Begin transaction

update UserProfile set rewards += reward where user_id = User;

update

End Transaction

- Notice the two transactions.
- **Consistency** between the tables is **not guaranteed**. In fact, a failure between the first and second transaction will result in the user table being permanently inconsistent.

Detour

Begin transaction

insert into eTransaction(tran_id, User, op, amount, description)

Queue message (Update Users, User, Reward);

Queue message (Update);

End transaction

For each message in Queue do

Begin transaction

update UserProfile set rewards += reward where user_id = User;

update

End transaction

End-for

- One can use queue like **message holders** to ensure correct updates.
- The modified program looks as follows.

Detour

- Notice the **decoupling** between the transactions corresponding to the eTranascation table and the Users table.
- Make the queue a **persistent** object so that updates are not lost.
- There may be one minor problem:
 - The queue has a message, but
 - Due to 2Phase Commit rules, the transaction is not committed!
- Solutions to this problem include
 - Transforming the updates to **idempotent updates**.
 - Record the list of applied updates separately. **Verify** each update against this list.

Detour

- The actions above indicate that we are **compromising** on the Consistency part of ACID semantics a bit.
- The database is always **available**.
- The state of the database is said to be **soft**.
- **Eventual consistency** is ensured because of other mechanisms.
- Therefore, partitioned databases try to support **BASE** semantics to increase availability.

Another Similar Setting

- Consider the Audio Alert box that vendors use.
- Think of the UPI transaction that will have the following style:
 - Debit User Account
 - Credit Vendor Account
 - Insert Audio record
- For the UPI, the third of these should be part of the entire transaction in principle.
- Can decouple this



Another Similar Setting

- Imagine that the debit, credit, and the queue entry are three subtransactions.
- Imagine a situation where the credit and the debit transactions are ready to commit but the queue entry fails.
- If the parent transaction requires that the queue entry is also equally essential for overall success of transaction, then the entire set of transactions should be aborted.
- This is not desirable since the user faces service disruption for no big reason.

Distributed Systems
Monsoon 2024
Lecture 16

International Institute of Information Technology
Hyderabad, India

Features of Distributed Database Systems

- We have studied a bit of how to relax transaction properties in distributed databases.
- We will now try to take a high-level view of various distributed databases.
- This helps us understand the design space and the important questions surrounding distributed databases.
- We will then study two example distributed database systems.

Features of Distributed Database Systems

- Data Representation
 - Two choices here, primarily
 - Use relational models
 - Use <key, value> pairs.
 - The former usually comes with additional aspects such as normalization, integrity constraints, foreign keys, and so on. This will require special operations such as joins to be able to answer queries.
 - The latter is more free-form. Allows for easy partitioning, scaling, caching, etc. One downside however is the inability to process complex transactions or complex queries.

Features of Distributed Database Systems

- Transaction Support
 - Varying degrees of support possible for transactions.
 - No support for multi-row transactions is an option.
 - Limited support subject to vagaries of distributed systems
 - Single row
 - Only within a partition
 - Hurdles
 - Inability to arrive at abort/commit decisions across partitions
 - Lack of consistent time hindering proper operations

Features of Distributed Database Systems

- Replication and Partitioning
 - Data to be scaled out necessarily to deal with the size challenge.
 - Several ways to partition, as seen earlier.
 - Replication is another mechanism to deal with data availability.
 - Comes with other challenges such as updates, consistency, location, number,

Features of Distributed Database Systems

- Data Consistency
 - With replicas and transactions, it is important to specify the consistency policy.
 - Several choices available here.
 - Updates at **Active** Location only: Designate one of the replica locations as an active location. All updates are processed only at the active location and percolated to other **passive** replicas. Can lead to inconsistency and availability issues
 - Update at **all locations**: All locations are active locations. Updates can be processed at any location and sent to other locations. Failure of replicas or updates can lead to inconsistencies.
 - **Majority** Rules: Update when a majority of the locations agree to the update.

Features of Distributed Database Systems

- Application Specific Systems
- Database systems built with specific applications in mind:
 - Documents: MongoDB, CouchDB
 - Graphs: Neo4j, HypergraphDB



One Example: DynamoDB

- DynamoDB is an improved version of SimpleDB that is a simple $\langle \text{key}, \text{value} \rangle$ store.
- Dynamo is designed to work on commodity hardware and focuses on availability, scalability, and reliability.
- Ensures that a large majority of requests have a very small latency.
- The latency aspect is critical for customer-facing applications.

DynamoDB

- Follows a workload-aware design.
- Most operations deal with a typical <key, value> mapping.
- Do not span multiple data items obviating the need for strong semantics of relational schema and data model.
- Interested in ensuring availability at the expense of consistency.
- This simplicity translates to a lot of design choices.

DynamoDB

- Dynamo aims to increase availability by using optimistic replication techniques where changes to data percolate to replicas over time.
- Additional writes are always accepted in a partitioned and disconnected network.
- Dynamo adopts an **eventually consistent** state where all updates reach all replicas eventually.

DynamoDB

- Given that Dynamo allows writes that are not always consistent, there is always the question of
 - how to resolve the conflicts, and
 - who resolves the conflicts.
- Dynamo, owing to the workload aware design model, empowers **applications** (workloads) to resolve conflicts based on the application semantics.
- Applications can also choose to push conflict resolution to Dynamo which uses generic and simple policies such as the latest write takes effect.

DynamoDB Architecture

- Dynamo offers several features and addresses several challenges including:
 - Partitioning,
 - load balancing,
 - scalability,
 - failure handling and recovery,
 - replication,
 - concurrency,
 - routing,
 - version management, and so on.
- Most of these are built on top of existing solutions.

DynamoDB Architecture

- Dynamo offers several features and addresses several challenges including:
 - Partitioning: Consistent Hashing
 - load balancing,
 - scalability,
 - failure handling and recovery: Gossiping and quorum systems
 - replication,
 - concurrency,
 - routing,
 - version management: Vector clocks
- Most of these are built on top of existing solutions.

The Dynamo Interface

- A very simple and minimal interface to applications with just two methods.
- The `get()` function takes a key as an input and returns an object or a list of objects with conflicting versions along with a context.
- The context of an object contains the metadata about the object including information on the version of the object.
- The `put()` function takes as inputs the key, value, and the context and updates the key with the provided value, updates the replicas, subject to the validity of the context provided as part of the input.
- The key and value fields are stored as byte arrays.

Dynamo: Data Placement and Partitioning

- Dynamo uses consistent hashing for data placement and partitioning.
- Dynamo makes a few modifications to the consistent hashing scheme.
- Each node is associated with multiple positions on the $[0, 1)$ ring by using multiple hash functions.
- Each such position is treated as a virtual node.
- Each actual node is now a collection of one or more virtual nodes.
- Virtual nodes brings two advantages:
 - provides for better load balancing
 - provide support for heterogeneity.

Replication

- For a configurable parameter N , Dynamo stores each object as N replicas.
- Each key k is stored at N nodes with one of them being the coordinator.
- The **coordinator** is the node that is **closest** to the hash of the key k in the clockwise direction.
- The other $N-1$ replicas are stored at the $N-1$ clockwise successor nodes to the coordinator node.
- If some of these nodes happen to be the virtual nodes that correspond to the coordinator, these are skipped to ensure that there are $N-1$ distinct **physical** nodes that act as replicas.
- The list of distinct physical nodes that store a given object is called as the **preference list** of the object.

Versioning

- Dynamo keeps multiple versions of each object in the system at the same time.
- Over time, older versions are subsumed (or overwritten) by newer versions.
- However, under certain circumstances such as concurrent updates and updates under failures, there could be multiple versions of the object that branch out from an older version.

Versioning

- Dynamo uses vector clocks to track the different versions of an object.
- The vector time of an object is the tuple of (node, counter).
- Each version of an object is timestamped with a vector clock.
- Given an object and two versions of the object, the properties of the vector clock allow one to see if the two versions are causally related or not.
- Versions that are not causally related need to be reconciled either by the application or by Dynamo.

Get and Put in Detail

- Assume for a moment that there are no intervening failures while these operations are under execution.
- The node that handles a get or a put operation is always the **coordinator** node.
- Even if the request arrives at a node in the preference list of the requested object, this request will be forwarded to the coordinator node.
- Dynamo uses the **quorum mechanism** to ensure consistency and availability.

Get and Put

- In a quorum system, there are three configurable parameters, N , R , and W that correspond to :
 - The number of replicas (N),
 - the size of the read quorum (R) ,
 - and the size of the write quorum (W), respectively.
- These parameters are to be chosen such that $R+W > N$.
 - Why?

Get

- To execute a get request, the coordinator requests all existing versions of the object from the N nodes in the preference list for that object.
- The coordinator waits to **receive at least** R replies.
- Once R replies are received, the coordinator **returns all versions** of the corresponding object that are not causally related.
- If there are indeed multiple divergent versions, the coordinator reconciles these divergent versions and writes a reconciled version that also superseded the latest version, thereby unifying the divergent versions.

Put

- A little more detail than Get.
- Each put request is tagged with a vector timestamp.
- The coordinator generates the vector time for the put event and writes the new value to the local store.
- The information about the new value along with the corresponding timestamp is sent to the $N-1$ nodes in the preference list.
- Each of these nodes send an acknowledgement to the coordinator.
- If the coordinator receives acknowledgments from at least $W-1$ other nodes, the write is deemed successful.

Failures: Get and Put

- Failures of nodes during the execution of the get and put operation are handled as follows.
- It is possible that the coordinator has failed.
- So, the operation is handled by the node at the top of the nodes in the preference list of the object.
- In the put operation, the update is sent to the top $N-1$ highest ranked reachable nodes in the preference list.

N, R, and W

- A low value of R and W assures very high availability but at the risk of increased inconsistency.
- On the other hand, a high value for R and W may render the service unavailable at times.
- Most applications using Dynamo set N to 3, and R and W to 2.